

JAVASCRIPT

1. BASICS OF JS

1. Introduction : JavaScript is a high-level, interpreted programming language used to make web pages interactive. It runs in the browser and is widely used for front-end and back-end development (Node)

2. Variables : variables are used to store data and can be declared using var, let & const.

- var : function-scoped, can be redeclared
- let : Block-scoped, cannot be redeclared
- const : Block-scoped, must be assigned a value at declaration

3. Data Types :

1. Primitive : String, Number, Boolean, Null, Undefined, Symbol, BigInt

2. Non-Primitive : Objects, Arrays, functions

```
let name = "John Doe";  
let age = 25;  
let isAdult = true;  
let marks = null;  
let city;  
let subjects = ["English", "Maths"]  
let Details = {  
    name: name;  
    age: age;  
}
```

4. Type Conversion & Type Coercion :

1. Explicit Conversion: Manually converting datatypes
2. Implicit Conversion: Coercion, automatic conversion of data types.

```
let n = "5";  
let sum = n + 10; // Implicit: Str + Num → Str  
console.log(sum); // "510"
```

```
let num = Number(sum); // Explicit  
console.log(num + 10); // 520 (Number)
```

We use functions like `Number()`, `String()` for explicit conversion of data types.

5. Operators In JavaScript :

→ Arithmetic Operators : perform mathematical operations

```
let a=10, b=5;  
console.log(a+b);    // 15  
console.log(a-b);    // 5  
console.log(a*b);    // 50  
console.log(a/b);    // 2  
console.log(a%b);    // 0  
console.log(a**b);   // 100000
```

→ Assignment Operator :

Assign values to variables.

```
let x = 10;  
x += 5; // x = x + 5  
x -= 2 // x = x - 2  
console.log(x); // 13
```

→ Comparison Operator : Compares value and return true or false.

```
console.log(10 == "10"); // true (loose comparison)  
console.log(10 === "10"); // false (strict comparison)  
console.log(10 != 5); // true  
console.log(10 > 5); // true  
console.log(10 >= 10); // true
```


→ Logical Operator : Combines conditions.

```
let isAdult = true, hasID = false;  
console.log(isAdult && hasID); // false (AND)  
console.log(isAdult || hasID); // true (OR)  
console.log(!isAdult); // false (NOT)
```

→ Bitwise Operator : Operate at Binary level.

```
console.log(5 & 1); // 1 (AND)  
console.log(5 | 1); // 5 (OR)  
console.log(~5); // -6 (NOT)  
console.log(5 ^ 1); // 4 (XOR)  
console.log(5 << 1); // 10 (Left Shift)  
console.log(5 >> 1); // 2 (Right Shift)
```

→ Ternary Operator : shorter way for if-else

```
let age = 18;  
let canVote = age >= 18 ? "Yes" : "No";  
console.log(canVote); // Yes
```

→ Nullish Coalescing Operator :

```
let name = null;  
let defaultName = name ?? "Guest";  
console.log(defaultName); // Guest
```

2. CONTROL FLOW

1. If-else if-else : Used to execute different blocks of code on conditions.

```
let age = 18 ;  
if (age < 18) console.log("You are a minor");  
else if (age === 18) {  
    console.log("You just became an Adult");  
} else {  
    console.log("You are an adult");  
}
```

2. Switch Statement : Used when multiple conditions depend on the same variable.

```
let day = 3 ;  
switch (day) {  
    case 1 : console.log("Monday"); break;  
    case 2 : console.log("Tuesday"); break;  
    case 3 : console.log("Wednesday"); break;  
    case 4 : console.log("Thursday"); break;  
    case 5 : console.log("Friday"); break;  
    case 6 : console.log("Saturday"); break;  
    case 7 : console.log("Sunday"); break;  
    default : console.log("Invalid Day");  
}
```

3. Loops in JavaScript :

- for loop :

```
for (let i = 1 ; i <= 5 ; i++) {  
    console.log(i);  
}
```

- while loop :

```
let count = 0;  
while (count < 3) {  
    console.log("Count", count);  
    count++;  
}
```

- do-while loop : (At least once)

```
let num = 0;  
do {  
    console.log(num);  
    num++;  
} while (num < 3);
```

- for-in loop : (Iterates over object properties)

```
let person = {  
    name: "John Doe",  
    age: 25  
}  
for (let key in person) {  
    console.log(key, person[key]);  
}
```


- for-of loop : (Iterates over Arrays)

```
let nums = [10, 20, 30];
```

```
for (let i in nums) {  
  console.log(i);  
}
```

4. Break and Continue:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) break;  
  console.log(i); // 1, 2  
}
```

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue;  
  console.log(i); // 1, 2, 4, 5  
}
```

3. FUNCTIONS :

1. Function Declaration :

```
function greet(name) {  
  return `Hello, ${name}`;  
}
```

2. Function Expression :

```
const greet = function (name) {  
  return `Hello, ${name}`;  
}
```

3. Arrow Functions (short syntax) :

```
const add = (a, b) => a + b;  
const sq = x => x ** 2;
```

4. Parameters & Arguments :

```
function add (a, b) {  
  return a + b; } → Parameters
```

```
console.log(add(5, 2));  
                → Arguments
```

```
function greet (name = "Guest") {  
  return `Hello, ${name}`; } → Default  
                                Parameter
```

```
console.log(greet()); // Guest
```

5. Callback Functions (Function passed as argument)

```
function something (data, callback) {  
  console.log(data);  
  callback();  
} → () => {Function}
```


4. OBJECTS

1. Creating objects :

```
const Person = {  
  name: "John Doe",  
  age: 25,  
  city: "Delhi"  
};  
console.log(Person.name); // John Doe.
```

2. Accessing Properties :

- Dot Notation (mostly used) :

```
console.log(Person.name); // John Doe
```

- Bracket Notation :

```
console.log(Person["age"]); // 25
```

3. Adding and Deleting Properties :

```
Person.country = "India"; // Add new property  
delete Person.city; // Deleting a property  
console.log(Person);
```

```
/* {
```

```
  name: "John Doe",  
  age: 25,  
  country: "India",
```

```
*, }
```

4. 'this' keyword: Refers to the current object

```
const user = {  
  name: "John",  
  greet: function () {  
    console.log(`Hello, ${this.name}`);  
  }  
};  
user.greet(); // Hello, John
```

5. Object.freeze() & Object.seal():

- freeze(): `Object.freeze(myObj)`; prevents modification & addition of new properties.
- seal(): `Object.seal(myObj)`; allows modification, but prevents addition of new properties.

6. Object Destructuring:

```
const student = {  
  name: "John",  
  grade: "A"  
};  
  
const {name, grade} = student;  
console.log(name); // John  
console.log(grade); // A
```

5. ARRAYS

1. Creating Arrays :

```
const fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits[0]); // Apple
```

```
const nums = new Array(1, 2, 3);  
console.log(nums); // [1, 2, 3]
```

2. Array Methods :

```
const arr = [1, 2, 3];
```

```
// Add element to the end
```

```
arr.push(4); // [1, 2, 3, 4]
```

```
// Remove element from end
```

```
arr.pop(); // [1, 2, 3]
```

```
// Remove element from start
```

```
arr.shift(); // [2, 3]
```

```
// Add element to the start
```

```
arr.unshift(1); // [1, 2, 3]
```

```
// Extract elements
```

```
arr.slice(1, 3); // [2, 3]
```

```
// Modify & Insert
```

```
arr.splice(1, 1, 4); // [1, 4, 2, 3]
```


3. Iteration over array

- forEach(): fruits.forEach(fruit => {
 console.log(fruit);
});

- map() [Returns a new Array]

```
const sq = nums.map(x => x**2);  
console.log(sq); // [1, 4, 9]
```

- filter() [Returns elements that match condition]

```
const even = nums.filter(x => x % 2 === 0);  
console.log(even); // [2]
```

- reduce() [Aggregates values into a single result]

```
const sum = nums.reduce((acc, num) => {  
    acc + num  
}, 0);  
console.log(sum); // 6
```

4. Spread Operator :

```
const arr1 = [1, 2, 3];  
const arr2 = [...arr1, 4, 5];  
console.log(arr2); // [1, 2, 3, 4, 5]
```

6. DOM

DOM is a programming interface that represents the structure of an HTML document as tree of objects, allowing JavaScript to interact with and modify the page dynamically.

1. Selecting Elements :

```
let title = document.getElementById("title");
```

```
let items = document.getElementsByClassName("items");
```

```
let divs = document.getElementsByTagName("div");
```

```
let abc = document.querySelector(".abc");
```

```
let efg = document.querySelectorAll(".efg");
```

2. Modifying HTML & CSS :

```
let heading = document.querySelector("#title");
```

```
heading.textContent = "New Title";
```

```
heading.innerHTML = "<span> Updated <span>";
```

```
heading.style.color = "blue";
```

```
heading.style.fontSize = "24px";
```

3. Adding or Removing Elements

```
let newDiv = document.createElement("div");
```

```
newDiv.textContent = "I am a new div";
```

```
document.body.appendChild(newDiv);
```

```
let removeElem = document.querySelector  
("#remove-me");
```

~~document~~

```
removeElement.remove();
```

4. Handling Events : Handle user interactions like clicks, keypresses, and form submissions.

- Click Event :

```
document.querySelector("btn").  
  .addEventListener("click", () => {  
    alert("Button Clicked");  
  });
```

- Keyboard Event :

```
document.addEventListener("keydown",  
  (e) => {  
    console.log("key pressed: ", e.key);  
  });
```

- Mouse Event :

```
document.getElementById("box").  
  .addEventListener("mouseenter", () => {  
    console.log("Mouse entered");  
  });
```